

7: Generation af tilfældige tal

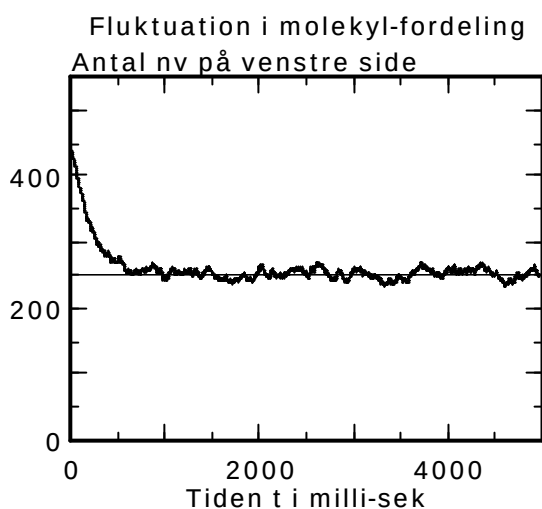
Fpro er udstyret med funktionen¹ $Random(x)$, der for givet positivt reelt tal x producerer et tilfældigt helt tal i det lukkede interval $[0, x - 1]$. Således vil $Random(6)+1$, såvel som $Random(6.27)+1$, producere de mulige udfald af et terningekast. Skriver man $Random(0)$, for man et tilfældigt reelt tal i intervallet $]0, 1[$. Nedenstående fig. 7.1 (med tilhørende tekst) viser et eksempel på dette. I fig. 7.2 vises et eksempel på brug af $Random(6)$

KLAR

```
t:=0
nv:=nv0
nh:=N-nv
SelectWindow(Graf)
Line(0,N/2,tmax,N/2)
```

LØKKE

```
p:=Random(0)
if p<nv/N then
  nv:=nv-1
  nh:=+1
else
  nv:=+1
  nh:=nv-1
endif
t:=t+1
If t>tmax then stop
```



Figur 7.1: Programmet *MAT07a.fpr*. Det illustrerer brugen af $Random(0)$ ved at se på følgende problem: Et antal molekyler N anbringes i en vandret liggende beholder, hvis indre er delt i en højre og en venstre del af en lodret væg med et lille hul i. Hvert millisekund passerer et molekyle fra den ene side til den anden. Det antal, der til tiden t befinder sig på venstre side, betegnes n_v . Antallet på højre side er så $n_h = N - n_v$. Hvorledes vil fordelingen udvikle sig i tidens løb, dersom vi til tiden $t = 0$ starter med et givet antal n_{v0} på venstre side?

Programmet lader en passage ske for hvert gennemløb af løkken og afgør retningen ved følgende sandsynlighedsbetragtning:

Sandsynligheden, for at en given passage sker fra venstre mod højre, må være $p_{v \rightarrow h} = n_v/N$, hvor n_v er bedømt på passagetidspunktet. Dette tal deler intervallet $]0, 1[$ i en venstre del $]0, p_{v \rightarrow h}[$ med længden $p_{v \rightarrow h}$ og en højre del $]p_{v \rightarrow h}, 1[$ med længden $1 - p_{v \rightarrow h}$. Sandsynligheden for, at det tilfældigt valgte tal $p = Random(0)$ ligger i venstre del, er altså netop $p_{v \rightarrow h}$. Derfor beslutter programmet, at hvis p ligger i venstre interval, så kommer molekylet fra venstre side, og løber ind i højre side.

¹ Strengt matematisk er der ikke tale om funktion - i en sådan skal samme input jo altid give samme output

KLAR

```

Proc Kvadrat(x,y)
  Line(x-0.5,y,x+0.5,y)
  Line(x+0.5,y,x+0.5,y+1)
  Line(x+0.5,y+1,x-0.5,y+1)
  Line(x-0.5,y+1,x-0.5,y)
endproc
SelectWindow(Graf 1)
ClearLines

```

```

For i:= 0 to N do
  n[i]:=Random(6)+1

```

```

Next

```

```

For i:= 1 to 6 do Hyph[i]:=0

```

```

i:=0

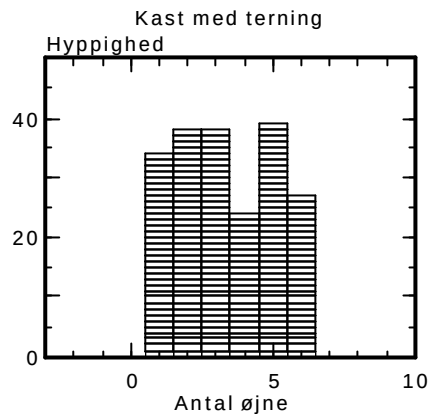
```

LØKKE

```

Kvadrat( n[i], Hyph[ n[i] ])
Hyph[ n[i] ]:=Hyph[ n[i] ]+1
i:=i+1
Pause(vent)
If i>N-1 then stop

```



	1	2
Tekst		Hyppig- hed
Navn	n	Hyph
Enhed		
Udtryk		
0	2	0
1	5	34
2	5	38
3	4	38
4	1	24
5	5	39
6	2	27
	3	

NB: Bemærk den her anvendte mulighed for at skrive
For - to - do løkken på een linje

NB: Denne ordre lader sig *ikke* simplificere til $Hyph[x]:=1$ ligesom $i:=i+1$ i denne linje er simplificeret til $i:=1$

Figur 7.2: Programmet *MAT07b.fpr*. Det illustrerer brugen af *Random(x)* ved at kaste en terning N (= 200) gange og skriver udfaldene i søjlen n . Navnet *Hyph* står for *Hyppighed*, og søjlen udfyldes, når programmet kører, således at $Hyph[i]$ kommer til at angive det antal gange, tallet i forekommer i søjlen.

Eneste egentlige forskel fra programmet *MAT6a.FPR* er, at denne gang er det ordren $Random(6)+1$, der producerer udfaldene i søjlen n .

NB: Programmet *kunne* simplificeres lidt ved i klar-delen at udelade den løkke, der udfylder n -søjlen, og istedet starte løkkedelen med linjen: $n[i]:=Random(6)+1$